

Python Keywords & Compiler vs Interpreter

🔍 *Understanding Reserved
Words and How Python Executes
Code*



➤ Today's Contents

📖 Topics Covered Today:

1. What are Python Keywords?
2. List of All Python Keywords
3. How to Identify Them in Code
4. Difference: Compiler vs Interpreter
5. Python Execution Flow
6. Practice & Quiz



🎧 “Know the language Python speaks, and how it speaks it.”



➤ What are Python Keywords?

🔑 Definition:

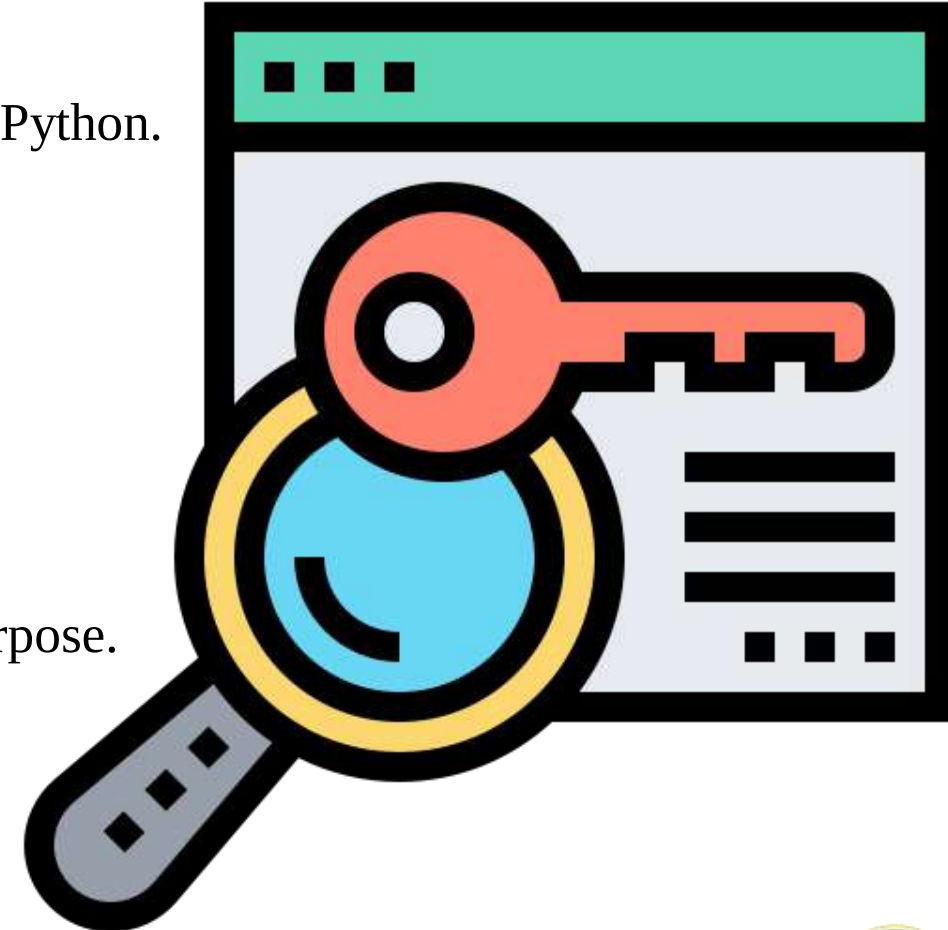
Keywords are **reserved words** that **have special meaning** in Python.

👉 You **cannot use them as variable names** or identifiers.

✦ Example:

```
if = 5    ❌ Invalid (because `if` is a keyword)
```

🔒 Think of them as **locked words** – Python protects their purpose.



List of All Python Keywords (Python 3.10+)

 **Total Keywords (as of Python 3.10): ~35**

☐  Shown in **groups** for better understanding:

Category	Keywords
Conditions	if, else, elif
Loops	for, while, break, continue, pass
Functions	def, return, lambda, yield
Classes	class, self, is, in
Boolean/Logic	True, False, and, or, not
Others	import, global, nonlocal, del, try, except, with, assert, raise, None, from, as, finally, await, async

 Use this code to list keywords:

```
import keyword
print(keyword.kwlist)
```



➤ Keyword Identification in Code

🔍 Sample Code:

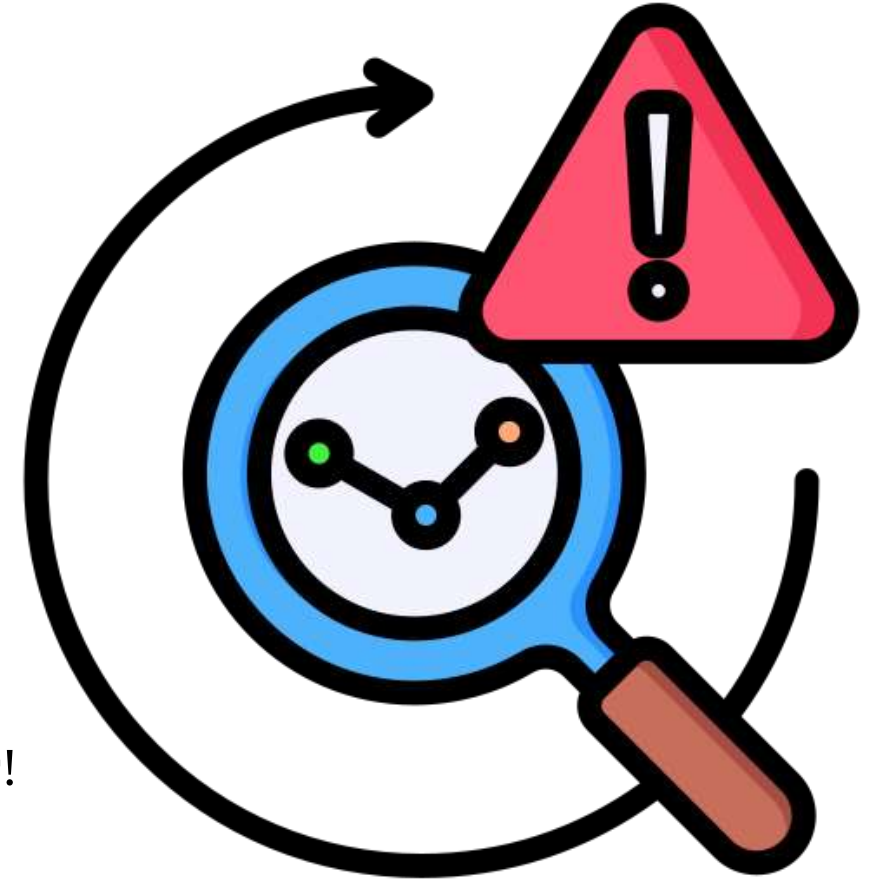
```
if age > 18:
    print("Adult")
else:
    print("Minor")
```

✓ Keywords used: if, else, print

📎 Tip: Keywords are usually highlighted in any code editor!

● Try this:

```
for = 10 # ❌ Invalid - 'for' is a keyword
```



➤ Keywords Are Case-Sensitive

True ✓ (Valid Keyword)
true ✗ (Not recognized)

Python keywords are always lowercase, except:

- True
- False
- None

Using the wrong case leads to error!



➤ Compiler vs Interpreter – What’s the Difference?

🛡️ Compiler vs Interpreter – Two Ways to Talk to a Machine

Feature	Compiler	Interpreter
Definition	Translates the entire source code into machine code at once.	Translates and executes code line-by-line or statement-by-statement.
Speed	Faster execution after compilation.	Slower execution due to real-time line-by-line translation.
Error Handling	Shows all errors after compilation is complete.	Stops at the first error encountered during execution.
Output	Generates an independent executable file (.exe, .out, etc.).	No separate file; executes directly.
Examples	C, C++, Java (uses compiler to bytecode)	Python, JavaScript, Ruby, PHP
Memory Usage	Uses more memory to store the compiled code.	Uses less memory as code is not stored in compiled form.



➤ Compiler vs Interpreter – What’s the Difference?

Feature	Compiler	Interpreter
Execution Time	Fast after compilation.	Slower due to interpretation at runtime.
Translation Time	Slower (complete code translation at once).	Faster (translates only required lines at runtime).
Portability	Less portable; compiled code is platform-dependent.	More portable; requires only the interpreter installed.
Best Use Case	Suitable for performance-critical applications.	Suitable for scripting, automation, and learning.

✦ **Python uses an Interpreter** ☐

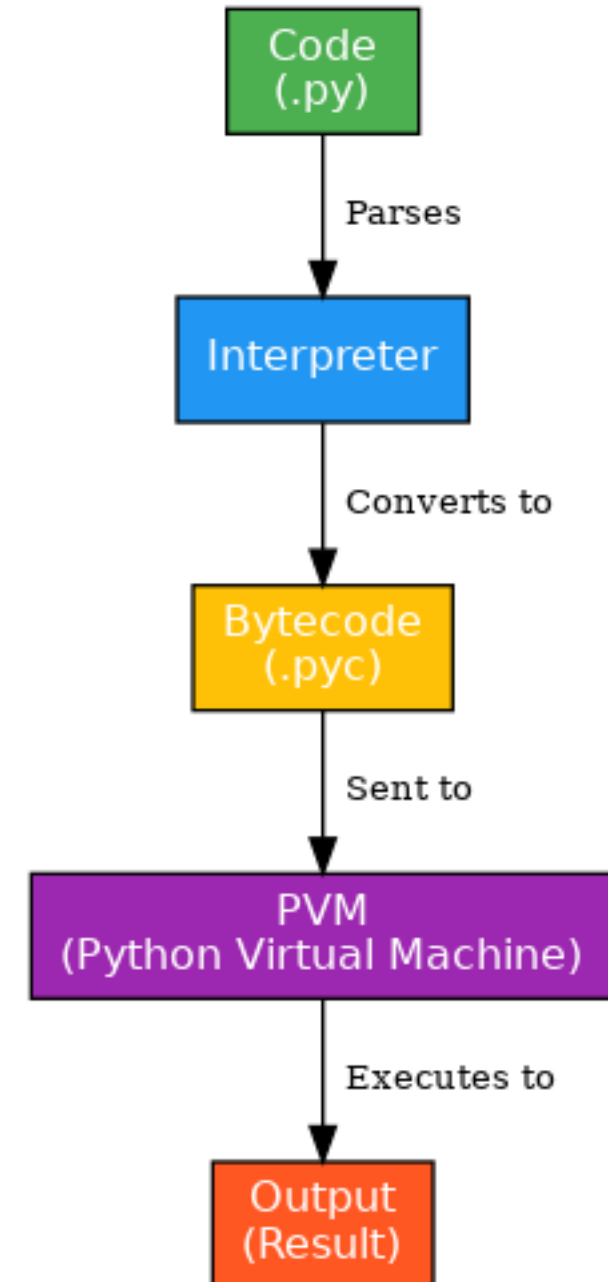


➤ How Python Executes Code

1. You write code in .py
2. Python **interpreter** reads and converts it to **bytecode**
3. Bytecode is sent to **Python Virtual Machine (PVM)**
4. Result is executed and shown

🔄 It's like:

☞ You (code) → Translator (interpreter) → Action (output)



➤ Analogy Time!

🧠 Compiler vs Interpreter Analogy:

Compiler	Interpreter
Full book translated before reading	Word-by-word translation while reading
Error found at end	Error found immediately
Less flexible	More flexible

📖 Think of **Compiler** like **book publishing**

🗣️ Think of **Interpreter** like **live translation**



Practice Time

 **Guess the Keyword:**

```
def greet(name):  
    print("Hello", name)
```

☞ Which words are Python keywords?

🎯 Challenge:

Try writing a program using 5 keywords!



